

GJØVIK UNIVERSITY COLLEGE

BACHELOR THESIS

Snowman

A complete, effective and secure rule management system for Snort

Authors:
Thomas NYHEIM
Eirik SKOGSTAD
Eigil OBRESTAD

Supervisor:
Slobodan PETROVIC

April 29, 2014

Preface

This is the preface.

Glossary

DOM Document object model.

HDD Hard disk drive.

HTML Hypertext markup language.

HTTP Hypertext transfer protocol.

IDS Intrusion detection system.

MD5 A cryptographic hash function producing a 128-bit hash value.

MVC Model-view-controller.

RAM Random access memory.

RPC Remote procedure call.

RPM Rounds per minute.

SSD Solid state drive.

SSL Secure sockets layer.

URL Uniform resource locator.

XML Extensible markup language.

Contents

Preface . . . . . **i**
Glossary . . . . . **ii**
Contents . . . . . **iii**
List of Figures . . . . . **vi**
List of Tables . . . . . **vii**
1 INTRODUCTION **1**
1.1 Document Structure 1
1.2 Terminology 1
1.3 Project Background 1
1.4 Previous Work 1
1.5 Target Audence 2
1.6 Project Objectives 2
1.6.1 Result Objectives 2
1.6.2 Effect Objectives 3
1.7 Project Description 3
1.8 Academic Background 4
1.9 Framework 4
1.9.1 Software Development Methodology 4
1.9.2 Schedule 5
1.9.3 Project Organization 5
2 EXISTING SOLUTIONS **6**
2.1 Functionality Comparison Chart 6
2.2 Bring Home The Bacon 6
2.3 Snortmanager 6
2.4 Other Programs of Note 6
2.5 Other Papers on Software Performance 6
3 REQUIREMENTS SPECIFICATION **7**
3.1 Functional Requirements 7
3.1.1 Program Workflow 7
3.1.2 Use Case Diagram 9
3.1.3 High Level Use Cases 10
3.1.4 Detailed Use Cases 13
3.1.5 Domain Model 15
3.2 Supplemental Requirements 16
3.2.1 Functionality 16
3.2.2 Usability 16
3.2.3 Reliability 16
3.2.4 Performance 16
3.2.5 Security 16
3.2.6 Interoperability 17

3.2.7 Licensing . . . 17
3.3 Constraints . . . 17
3.3.1 Platform . . . 17
3.3.2 Data security . . . 17
3.3.3 Graphical User Interface . . . 17
4 DESIGN . . . 18
4.1 Server . . . 18
4.1.1 Architecture . . . 18
4.1.2 Database . . . 19
4.2 Client . . . 20
4.2.1 Architecture . . . 20
4.2.2 Database . . . 22
5 IMPLEMENTATION . . . 24
5.1 Programming languages . . . 24
5.1.1 Standards and guidelines . . . 24
5.2 Development environments . . . 24
5.3 Frameworks . . . 24
5.4 Concepts of Snort . . . 24
5.4.1 Rules . . . 24
5.4.2 Rule sets . . . 24
5.4.3 Generators . . . 25
5.4.4 References . . . 25
5.4.5 Event and detection filters . . . 25
5.5 Core module . . . 25
5.5.1 Rules and rule revisions . . . 26
5.5.2 Rule sets . . . 26
5.5.3 Sensors . . . 26
5.5.4 Comments . . . 26
5.6 Update module . . . 26
5.6.1 Sources . . . 27
5.6.2 Files . . . 27
5.6.3 Running an update . . . 28
5.6.4 Rule parsing . . . 28
5.6.5 Classification, Generator and Reference Type parsing . . . 29
5.6.6 Logging . . . 29
5.7 Graphical User Interface . . . 29
5.7.1 Structure . . . 29
5.7.2 Design . . . 30
5.7.3 Overview of central GUI features . . . 30
5.8 Logging . . . 34
5.9 Configuration file . . . 35
5.10 Documentation . . . 35
6 SOFTWARE TESTING . . . 36
6.1 Strategy . . . 36
6.2 Description of problems . . . 36
6.2.1 List pagination . . . 36

7 PERFORMANCE TESTING **37**
7.1 Definitions 37
7.2 Environment 37
7.3 Test suites 37
7.3.1 Update processing 38
7.3.2 Rule distribution 39
8 RESULTS **40**
9 ANALYSIS OF RESULTS **42**
10 CONCLUSIONS **43**
Bibliography **44**

List of Figures

1 Program Workflow 7
2 Program Modules 8
3 Use Case Diagram 9
4 Conceptual Class Diagram 15
5 Server Architecture 19
6 Core server-database 20
7 Tuning server-database 21
9 Client Architecture 21
8 Update server-database 22
10 Client Database 23
11 Flow 25
12 GUI Structure 30
13 Rules page 30
14 Rulesets page 32
15 Update progressbar 32
16 Update changes 33
17 Form validation 34

List of Tables

10 Effects of different filter types. 26
11 Rule options used in this system. 27
12 Test systems. 37
13 Test cases in Test 1. 38
14 Test results for Test 1 and 2. 40
15 Test results for Test 3. 41
16 Test results for Test 4. 41
17 Test results for Test 5. 41

1 INTRODUCTION

1.1 Document Structure

How have we organized this document.

1.2 Terminology

Open Source Software - Software where the program code is publically available and is licensed in such a way that the copyright holder provides the rights to study, alter or distribute the software to anyone for any purpose [1].
Daemon - A computer program that runs as a background process, usually without any user interaction.
Sensor - A computer system wherein Snort and/or other security software is recording network traffic and produces alerts when malicious content is detected.

1.3 Project Background

The Norwegian Armed Forces Cyber Defense (CYFOR) unit for Computer Network Defense (CND), Avdeling BKI, is responsible for detecting and stopping cyber-attacks against the critical infrastructure and command and control systems utilized by the Norwegian military. One of the tools used to carry out this task is the open source Intrusion Detection System (IDS) Snort [2].

Snort is a leading open source IDS that is used to detect potentially unwanted or malicious network traffic based on predefined patterns described in signatures. There are both free and commercial signatures available for Snort and as Snort has been highly adopted by the security community, these signatures are continually updated and maintained. Using Snort has been plagued for years by the fact that it does not have a good system for handling, updating or maintaining the signatures (henceforth called rules and rule sets) used for detection, thus making it cumbersome to utilize Snort for security analysts.

Avdeling BKI has therefore tasked a group of students from GUC to design and develop a more complete, effective and secure rule management system for Snort that can continue to live on beyond this project.

1.4 Previous Work

Research [3] has shown that a good rule management system for Snort will improve the workflow and efficiency of the IDS significantly and a prototype for such a system was built in 2012 called Bring Home The Bacon (BHTB) [3]. This prototype has been in use at Avdeling BKI since 2012, but is suffering from lacks in functionality and efficiency due to being a proof-of-concept prototype. The prototype did, however, prove that there is a significant gain from having an intuitive Graphical User Interface (GUI). The operators workflow becomes much more efficient, as you go from having to do complex operations in a Command Line Interface (CLI) to just pushing buttons to make the system do things automagically.

There was another similar project conducted at Gjøvik University College (GUC) in 2012 called Snortmanager [4] that also deals with managing Snort rules. However, that project was not properly released as open source software. The project files are available but there is no documentation or instructions publically available that describes usage and installation. The project has also not been maintained since its initial release and is designed for a specific company in mind.

Enabling, disabling and configuring rules is also cumbersome in this system as it is based on plaintext arguments in "policies" that users must construct themselves. The rule distribution is also done manually from the files generated. This makes Snortmanager something not very usable for Avdeling BKI and their routines.

There are also other software available in the Snort community that work very well for that they do, but are limited in their scopes are more suited for smaller, single sensor implementations of Snort. Good examples of these are Pulled_Pork[1] and Oinkmaster[2].

The key differences between all these pieces of software and Snowman will be explained in more detail in chapter 2.

1.5 Target Audence

The software this project has produced is targeted at users of Snort as an IDS. More specifically, this target audience usually consists of security experts with a high degree of knowledge of Snort and computers in general.

This report is targeted at those that may be interested in discovering just how complex the rule structure of Snort really is, and academics that are interested in reading about software complexity versus efficiency.

1.6 Project Objectives

1.6.1 Result Objectives

We reiterate from the project plan[link to project plan] that this project has a goal of researching, designing and developing a system that as a minimum:

- Is more efficient than BHTB.
- Is developed as an engine that responds to API calls from any arbitrary user interface.
- Is modular in design and allows future changes and additions to not affect existing functionality.
- Is scalable enough to handle both small and large sets of IDS sensors and rules in a production environment.
- Is able to download rules and rule sets from multiple customizable sources.
- Is able to distribute different rules to different IDS sensors.
- Can download and distribute rules automatically at set intervals.
- Is capable of exporting rules for sharing.
- Has the capability to activate, deactivate, filter and suppress rules on a per sensor basis, in a way that is user friendly.

[1] https://code.google.com/p/pulledpork/
[2] http://oinkmaster.sourceforge.net/

- Supports multi-user interaction.
- Can efficiently and safely work with different installations of Snort and various other third party tools, such as SNORBY[3].
- Is easy to install.
- Has an internal logging system.
- Is well documented and can be easily adopted by anyone as open source software.

In addition, the project will attempt to add the following functionality if time permitting:

- A graphical user interface.
- Said user interface must be user friendly and intuitive, and require only limited knowledge of Snort rules from the user.
- The system can also distribute rules to Suricata[4], another Snort-like IDS, which can operate with the exact same rules as Snort.
- Implement security features that ensure confidentiality of rules.
- The project will investigate and potentially implement multi-threading technology or other optimization techniques to exploit modern hardware capacities.
- The system is capable of validating and checking rules for errors before they are distributed to sensors.
- The system supports commenting of the rules.
- The system supports writing custom rules and editing existing rules.

Once the system has been developed, it will be released as an Open Source Software to the public in a well-documented format.

1.6.2 Effect Objectives

Forming baselines from the current prototype BHTB and from Snortmanager, the project has a goal to further increase the efficiency of rule management by investigating and implementing programming techniques such as multi-threading and a change-based system to reduce latency and delays in the system by as much as 50%.

As a very minimum the software efficiency must be linear or better, meaning if 1 action takes 1 second, 10 actions must take 10 or less seconds.

The project will therefore be measuring for these effects.

1.7 Project Description

In broad strokes, this project has been about designing a system that provides a simple and efficient way of handeling rule management for the Snort IDS. The project is the spiritual successor to BHTB. Essentially taking good ideas from BHTB, adding a few of our own, and creating a more adoptable open source rule management system for Snort.

We utilized the latest stable version of Python as the main programming language for the project, which is version 2.7. C++ was also considered, but due to the limited time at our disposal, we felt we could create more functionality with Python, due to it requiring

[3] http://snorby.org
[4] http://suricata-ids.org

less memory management and the general “programmer friendly” ways of doing things.

The system required a database to function efficiently and the project primarily utilizes MySQL, as it is the de facto standard for the Linux platform. Furthermore, the project also consisted of creating a GUI that is user friendly and responsive.

The software that was created in this project, though still lacking some features and contains some minor bugs, largely fulfils the project goals outlined above. We did not fulfil the goal of producing software that has reduced latency and delays, but we are confident that this is because we have sacrificed efficiency for functionality. We also feel that the functionality gained outweighs the efficiency lost and the efficiency curve is still very much within the linearity objective, as you can read about in chapter **??**.

Because the project has been so performance oriented, the report has evolved to also contain a small study into software performance and it’s relationship with software complexity.

1.8 Academic Background

Talk about our pasts.

1.9 Framework

1.9.1 Software Development Methodology

As this project has been very time-restricted, the development should be as efficient as possible. We therefore needed a model that utilized the manpower we had available in the best possible way. The size of the group was quite small, so big and complex development models would add too much bureaucracy to coordinate the project and was thus overkill.

As we wanted to have a lightweight development-model, some form of an agile development model with well-defined development-cycles was preferred. This way, each group-member could take an active part in the project and all members could feel a sense of ownership.

Developing iterative and incrementally, we got the best from both the unstructured world, and from the very well defined waterfall approach. We were able to get feedback during the project, and have a structured and reasonable way of actually doing changes to the plan while the project was running. At the same time we could be able to maintain the overall plan within cycles, so that it was possible to work individually, while still being well coordinated.

To be able to develop all the modules of the project within the timeframe, each development cycle consisted of multiple modules developed in parallel. This allowed us to maximize our utilization of time and personnel.

The start of each cycle consisted of a meeting where we defined the goals for that cycle, and identification of the individual tasks that we needed to get done. This is similar to the start of a sprint in the development model Scrum.

In the development part of the cycle, we individually worked on the tasks that were defined in the planning-part. Regular meetings and cooperative programming within the group would ensure that we all pulled the project in the same direction.

At the end of each cycle we would have a working set of software, which ideally complied with all the goals set in the start of each cycle. We planned to have meetings with Avdeling BKI between each cycle to present the progress, and to let them influence

the direction of the project.

After all development cycles had been completed, we then made a final assurances that everything worked as intended and that all modules coexisted in harmony. This part of the development also consisted of hands-on testing by the client and various performance testing.

1.9.2 Schedule

We planned this project to have a fairly comprehensive initial phase for mapping and designing of the system. We spent the better part of the first two weeks figuring out how to implement the system. After the two weeks, we went to our client and presented our work for their feedback and a final go-ahead before we began programming.

We then began the implementation phase which consisted of development cycles as described above. This phase was devided into five parts, consisting of four development cycles and a final cleanup/testing cycle. All the development cycles lasted two weeks, aside from the last development cycle and the cleanup/testing cycle, which lasted one week each.

During those cycles we conducted meetings with both our mentor and our client every two weeks, timed so that these meetings took place on alternating weeks. This ensured that we got continual feedback and advice throughout the entire process.

After these weeks, we completely shifted focus from the software to the project report and spent the final 5-6 weeks working on that. Because we had done a thorough job during the first few weeks of the project, we had a lot of the contents for the report more or less created.

1.9.3 Project Organization

We organized the project work a little differently throughout the project, reflecting that we had multiple phases. During the first few weeks we spent most of the time together in a group room, working together and discussing things. Once most things had been planned out and the implementation phase began, there was more room and need to work individually to produce code. We still gathered together every so often to syncronize our progress and plan the next course.

Throughout the project, most decisions have been met either by confering with the client or achieving consensus within the group. At no point during the project has there been a need to enforce the project rules.

2 EXISTING SOLUTIONS

To identify exactly what Snowman does differently than other, similar software, we will quickly go over their functionality and how they differ.

2.1 Functionality Comparison Chart

put inn tabell over funksjonalitet i de forskjellige programmene

2.2 Bring Home The Bacon

Beskriv hva den gjør

2.3 Snortmanager

Beskriv hva den gjør

2.4 Other Programs of Note

Beskriv hva den gjør

2.5 Other Papers on Software Performance

*** Trekk inn hva andre har skrevet om det ***

3 REQUIREMENTS SPECIFICATION

Many of the requirements for this system stems from functions that already reside in BHTB, which our client wants either reproduced or improved upon. That means the requirments for this project is closely tied to the way BHTB operates and its general workflow. In this coming section we will outline the specifications we've gathered from BHTB and from conversations with the client.

3.1 Functional Requirements

3.1.1 Program Workflow

To develop our requirements, we decided to chart the program workflow in order to better understand what the system is expected to do. Since this was to be a highly user-oriented system, we accomplished this by going over the old system, BHTB, step by step with the client and detailing the functions and various needs for improvement or additional functionality. From this, we developed a workflow chart that reveals the major system parts.

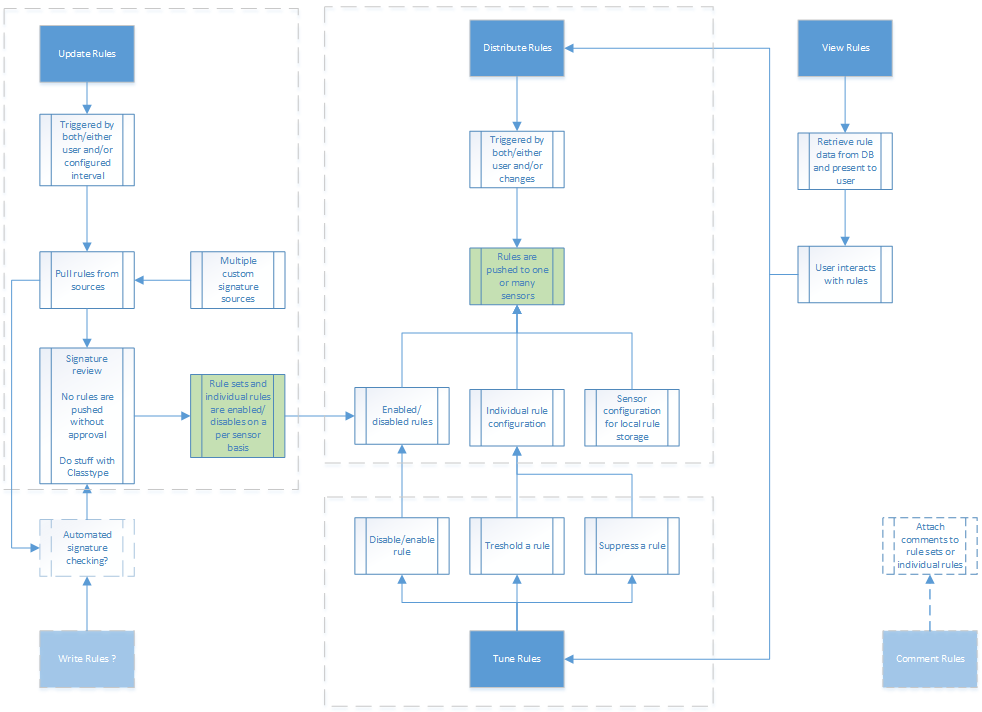


Figure 1: Program Workflow

To explain the workflow chart in figure 1, the blue boxes are the major system functionalities, the green boxes are the major goals of the system and the arrows show the direction of flow the system should have.

As you can see, if you trace the arrows, all of the program flow ends up with rules being syncronized to the sensors. This, of course, makes sense since that is the whole purpose of the system to begin with.

But the road to Rome consists of a lot more functionality that needs to be in place, and that interacts with the rules, before they are ready to be handed over to Snort.

It's important to note that very early on, we decided that writing rules, automatically check them for errors and commenting them were nice-to-have (but still important) features that we could implement if we had time to do so. Hence they are greyed out in the drawings, and as you will notice further on in the report, wes never developed due to time contraints.

From this workflow we could outline a rough modular design (figure 2) based on the naturally separate functions of updating rules, tuning them, distributing them and show them. We also added a database module, since most of our program was going to be centered around it.

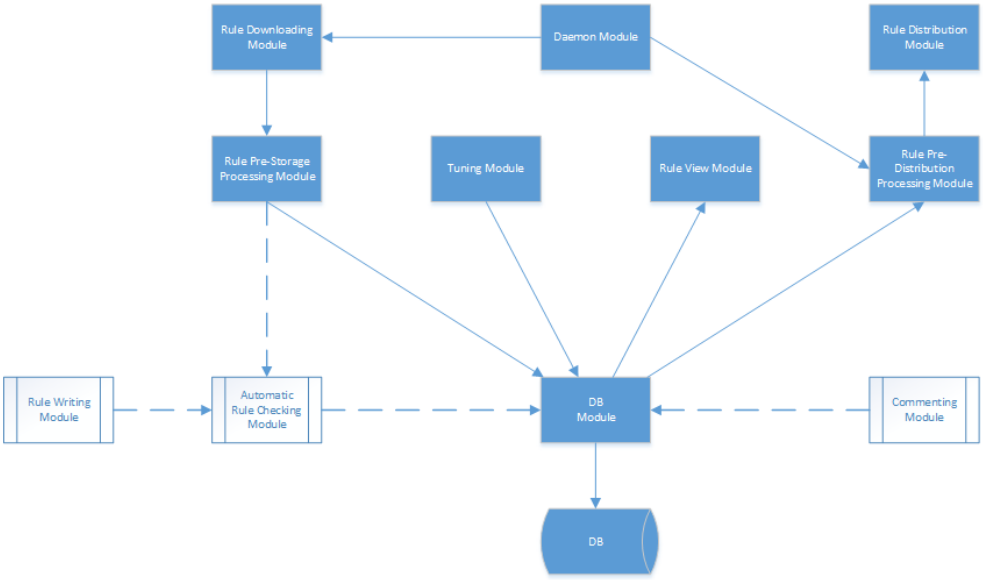


Figure 2: Program Modules

Rule Downloading Module

This module will handle all functionality related to downloading rules, including comparing MD5 checksums.

Rule Pre-Storage Processing Module

This module will essentially take downloaded rules, and parse and process them into objects that can be fed into the database.

DB Module

This module will handle all input and output to/from the database.

Rule Pre-Distribution Processing Module

This module will essentially take rules stored in the database and prepare them for distribution to sensors.

Rule Distribution Module

This module will be in charge of distributing rules out to the Snort sensors and handing the rules over to Snort itself.

Tuning Module

This module will handle all functionality related to turning rules on or off, filtering them or suppressing them.

Rule View Module

This module will contain all the functionality needed to display the system status and system functions to the user. It will be the primary way the user interacts with the system. Basically the GUI module.

Daemon Module

This module will run continually on the system to ensure that the system is responsive and to carry out any interval based tasks, such as automatic updates or distributions.

Commenting Module

This module will allow users to attach comments to rules.

Rule Writing Module

This module will handle functionality for writing custom rules and editing existing rules.

Automatic Rule Checking Module

This module will be responsible for verifying rules to ensure that there are no malformed or misconfigured rules that may potentially break or cause a halt on a Snort sensor.

3.1.2 Use Case Diagram

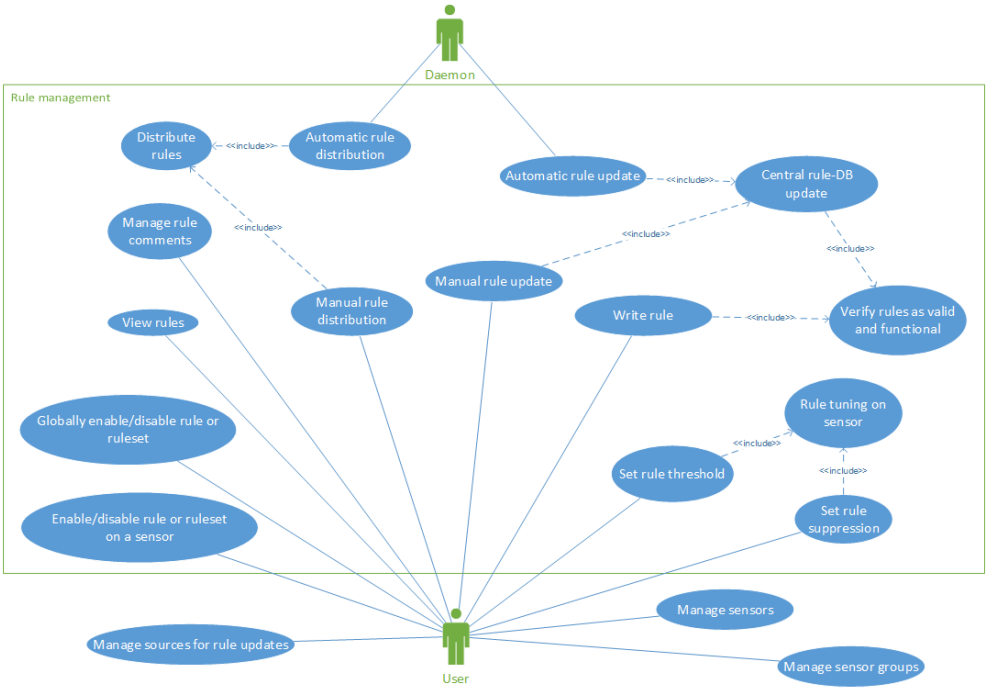


Figure 3: Use Case Diagram

3.1.3 High Level Use Cases

Looking at the arrows in the program workflow in figure 1, it is pretty apparent that the program workflow is heavily centered around the database module, and the major parts of the system revolves around the importing of rules and then distributing them. We therefore began developing our use cases around those major movements.

We've added the most central use cases for this system below, but additional use cases is available in appendix B.

Use case	**Manual rule update**
Primary actor	User
Purpose	Update the central rule database with the newest rules and revisions.
Description	The user commands the server to update the rules, either from a local file or from external sources. When the latter is the case, the server pulls rule files from all the sources specified in the list of sources. In both cases rules are processed and stored in the database.
Preconditions	Either a file or one or more sources must exist.
Postconditions	The central rule database is up to date with the newest rules and revisions.

Use case	**Automatic rule update**
Primary actor	Daemon
Purpose	Update the central rule database with the newest rules and revisions.
Description	At configured time-intervals, the system daemon triggers a rule database update. The server pulls rule files from all the sources specified in the list of sources to be updated at the given interval, processes the rules and then stores them in the database.
Preconditions	One or more sources must exist.
Postconditions	The central rule database is up to date with the newest rules and revisions

Use case	**Manual rule distribution**
Primary actor	User

Purpose	Distribute rules to sensors as per the settings in the database.
Description	The user sends a command to distribute rules to either all or specified sensors. The server then begins the distribute routine, and eventually all sensors will be up to date with the rules and rulesets they should have.
Preconditions	Rules and sensors must exist
Postconditions	Specified sensors contain rules as per settings in the central database.

Use case	**Automatic rule distribution**
Primary actor	Daemon
Purpose	Distribute rules to sensors as per the settings in the database.
Description	Configured at intervals, the system triggers a command to distribute rules to specified sensors. The server then begins the distribute routine, and eventually all sensors will be up to date with the rules and rulesets they should have.
Preconditions	Rules and sensors must exist
Postconditions	Specified sensors contain rules as per settings in the central database.

Use case	**Enable/Disable a rule or ruleset**
Primary actor	User
Purpose	Enable or disable one or more rule or ruleset on one or more sensors.
Description	The user either enables or disables a rule or a ruleset so that the rule is either on or off on certain sensors. This status is then shared with the sensor and the sensor syncronizes as needed.
Preconditions	Rules and/or rulesets must exist.
Postconditions	A rule or ruleset has been modified to be enabled or disabled and the change is syncronized to affected sensors.

Use case	**Set rule threshold or suppression**

Primary actor	User
Purpose	Set or remove a threshold or suppression for a rule on one or more sensors.
Description	User sends a command with either thresholding/suppression parameters or removal of a threshold or suppression for a rule on one or more sensors. The system updates the database and distributes the change to the sensor(s).
Preconditions	Rules and sensor(s) must exist.
Postconditions	Rule tuning is saved in the central database and distributed to affected sensors.

Use case	**View rules**
Primary actor	User
Purpose	Display rules to the user.
Description	Rules are retrieved from the database, processed for GUI format and then delivered to the GUI.
Preconditions	Rules must exist.
Postconditions	The user is informed of the current status of the central database and the status of rules and rulesets enabled/disabled on sensors.

3.1.4 Detailed Use Cases

We also worked on some detailed use cases on our more complex system modules. Again, more use cases are available in appendix B.

Use case	**Central rule-database update**
Primary actor	Server
Purpose	Update the central rule database with the newest rules and revisions.
Description	An update can be triggered either by the user or the daemon. The system retrieves the files in question, checks what is new and what is not and then updates the database accordingly.
Preconditions	Depending on the request, this process requires a local file or the server must be configured with external source(s).
Postconditions	The central rule-database is up to date with the latest rules and revisions.
Triggers	Manual rule update, Automatic rule update, Write rule

Basic Flow:

1. Update is triggered.
2. Rule Downloading Module compares hash values from external sources with last update.
3. Updated rule files are downloaded from sources. The files are validated and passed to the Pre-Storage Processing Module.
4. The new files are parsed for rule SID and revision number.
5. A list of rule SID and revision numbers are gathered from the database and these are compared to identify which rules are new or modified.
6. New or changed rules are then parsed into objects and these objects are passed to the Database/Core Module.
7. The Database/Core Module stores the new or changed rule objects into the database. The user is notified that the update is complete and given a list of changes.

Extensions:

1. 1. Update is triggered by user.
   2. Update is triggered by daemon.
3. 1. The hash-values from all sources are identical with previously stored values. Database is up to date and the update process terminates. User is notified that database is up to date.
   2. The hash value from one or more sources is newer than previously stored values. Database must be updated, and the user is notified that the database is not up to date and an update process has begun.

Error handling:

1. The source and database checksum is different but the source rules are older than what is stored in the database: *The program can still continue, because if the SIDs already exist they will not be added again and if the revisions are older the rules will also not be updated.*
2. A ruleset has changed its name or one or more rules are moved to a different ruleset: *Based on a configuration file, the affected rules will either be a) automatically moved to the new ruleset or b) stay put in their original ruleset. The user will be notified and can choose to reverse or distribute the changes in either case.*

Use case	**Distribute rules**
Primary actor	Server
Purpose	Distribute rules from the central database to the sensors.
Description	After a command to synchronise is sent either by the user or the daemon, the server calculates and organizes the central database data and synchronizes this information with the affected sensors.
Preconditions	One or more rulesets or rules must be present in the central database. One or more sensors must be registered.
Postconditions	The rules on the sensor(s) are synchronized to reflect those in the central database that are enabled/disabled for those sensors.
Triggers	Manual rule distribution, Automatic rule distribution

Basic flow

1. Synchronize signal is sent.
2. The system calculates and organizes the data for each sensor affected by the synchronization.
3. The system initializes a synchronization protocol with the affected sensor or sensors, sending a list of what the sensor should have.
4. The sensor calculates diff between should have and has.
5. Compare diff.
6. Send update.
7. The sensor includes the new changes into its database and files and tells the central system that it is now up to date.
8. The system carries out maintenance to reflect the synchronization.
9. The user is notified that all sensors affected are now up to date.

Extensions

1. 1. User sends a command to synchronize one, more or all sensors
   2. The system reaches an interval for synchronizing all sensors.
   enumi4
3. 1. The sensor content is different from what it should have, asks central system to send what it needs.
   2. The sensor content is not different from what it should have, no further action is required. This status is sent to the central system.
3. 1. The system retrieves and sends what the sensor asks for.
   2. The system terminates the synchronization protocol with the sensor in question.

Error handling

1. The change may never be synced to the sensor due to errors: *There must be an error protocol that is followed in this scenario.*

3.1.5 Domain Model

After outlining all the things above, we put together a concept for the objects we would need in the system. Even though a Snort rule is quite simple in and of itself, there are a lot of meta information surrounding to them, like rule revisions and rule classes. Thus our system needs at least 16 different classes in order to fullfil the basics of the required functionality.

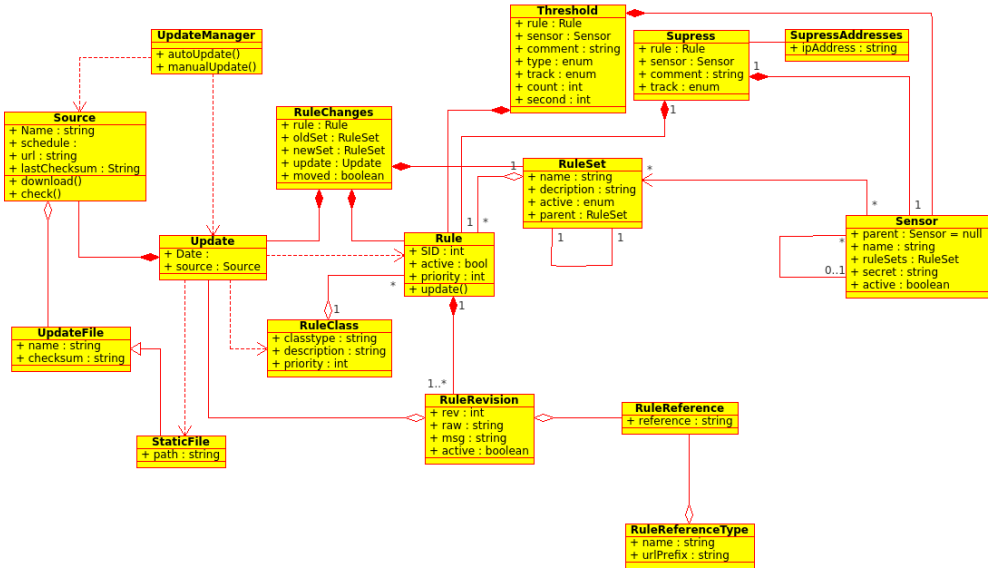


Figure 4: Conceptual Class Diagram

3.2 Supplemental Requirements

In addition to the features discovered in the use cases, the system has some requirements that can be broken down into a RUP FURPS+[1] fashion.

3.2.1 Functionality

- The system should be able to restart a Snort sensor in the event that one crashes.
- The system should be able to detect if a Snort sensor is not responsive over the network.
- The system must be portable to the majority of the popular linux distributions.

3.2.2 Usability

- The graphical user interface must be simple and clean, but still contain all the features any user might require to interact with the back-end system.
- The system interfaces, herein API and function calls, must be self-explanatory.

3.2.3 Reliability

- The system must not cause a SNORT sensor to crash.
- The system must be responsive to API calls at all times. In the event the system capacity limit is reached, new requests should be queued.

3.2.4 Performance

- The system should handle administration of at least 200 sensors.
- The database should be able to maintain at least 100 000 individual rules.
- Any operation in the system in normal operations (i.e. after the first big rule update) should take no longer than 1 second on average.
- The relationship between time and workload should be linear, meaning that if a single operation that takes 1ms, ten operations should take 10ms.

3.2.5 Security

- The link between the system server and the remote sensor must be secured in such a way that a man in the middle cannot read the transmission in plaintext.
- The remote sensor and the system server must verify that they are trusted parties in the system, through authentication.
- Certain important API calls must be protected against misuse from a non-trusted party.
- All processes and files related to them system must exert the principle of least privilege.

[1] A more comprehensive version of the usual requirements list. Described here: http://www.ibm.com/developerworks/rational/library/4706.htmlN10073

3.2.6 Interoperability

The system must be able to work with Snort in such a way that it can import new files to Snorts configuration files structure and must be able to reset Snort in such a way that the new configurations are loaded.

3.2.7 Licensing

The system will be released to the public as Open Source under the GNU General Public License v3 license[2].

3.3 Constraints

3.3.1 Platform

As mentioned in the Project Plan, Snort is primarily used in conjunction with the Linux/UNIX operating systems. For that reason, the project was restricted to only develop for that platform. More specifically, we limited ourselves to the Ubuntu and Debian operating system, which is the primary platform the client uses as well. We have, however, stived to make the software portable.

3.3.2 Data security

While the system itself will not handle any data that may comprise of personal data, it may contain rules and rule sets that are considered classified or confidential. The system has therefore been built with this in mind and ensures confidentiality within the boundaries of the system. The system does not guarantee the confidentiality of the system outside these boundaries.

These boundaries are defined as the program and its processes and threads, and the network link established between the system and remote sensors. The system assumes that the users have already established database security.

3.3.3 Graphical User Interface

The project has had its main focus on the systems functionality and efficiency, and developing a graphical user interface (GUI) has been considered a secondary task and was prioritized as such. None the less, a GUI for demonstrational purposes has been included.

[2] http://www.gnu.org/licenses/gpl.html

4 DESIGN

This chapter will present the design specification of Snowman. Snowman is developed as a distributed system (why?), with a central server which is responsible for the user-interface and the central core functions such as distributing rules to the sensors and downaload updates from external sources. A Snowman client, responsible for the communication with the central server, is installed on each sensor. The principal tasks of the client is to generate Snort rule files and respond to the server's status requests.

4.1 Server

The server is the central component of Snowman and generally consists of:

- Central rule storage
- Web-interface for user interaction
- Update-module to get updates from rule-sources
- RPC-interface to communicate with the clients

The central storage of rules would be a normal relation database. As we have decided to use Django as our framework, we have a great flexibility to which database we actually use. In the current version of Django (1.6) we might use MySQL, PostgreSQL, SQLite or Oracle.

For the RPC-Interface, we have decided to use XML-RPC. This is a very simple protocol using XML over HTTP to transfer requests and data to and from the server. To be able to keep an aspect of privacy, we encrypt the raw HTTP stream using SSL.

4.1.1 Architecture

The server architecture follows the layered approach of MVC (figure 5).

The web-interface uses pure Django logic. The HTML visible in the users browser is generated by the Django template-engine, which recieves its data from the views created by using ordinary pyhton code.

The XML-RPC server uses a handler which is administering the connections from the sensors, and exposing the relevant methods to allow the sensors do a synchronization.

The update is unpacking and preparing the update-files for parsing. It then handles the files to the parser. The parser is then handling the different parametres to the Updater, which in turn saves them to the database in an efficient manner.

Finally, common for all the interfaces is the Django Models. We use the database-abstractionlayer from Django for mapping the lines of a relational-database to django objects. This is the part that lets us write code once, and still be able to use several different database-engines.

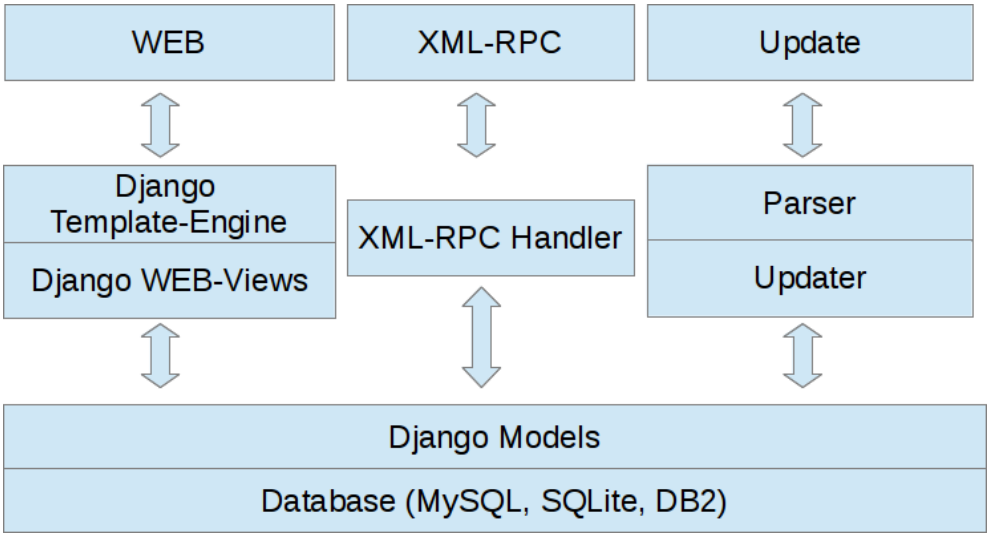


Figure 5: Server Architecture

4.1.2 Database

As the database of the server is quite complex, we have divided the drawings, so that we have one drawing per module.

Core

The core database-architecture used for storing the rules is as following:

The most central part of our system is the representation of a Rule. A single Rule might have several revisions, as we might want to keep some history when they get updated. Each revision have its own set of references attached.

A single rule also have a lot of other properties which it might share with other rules. A generator, a classification and a ruleset are examples of such.

Finally, the rules are placed onto a sensor by attaching ruleSets to the sensors.

Tuning

In addition to storing the raw rules, we would like to be able to store tuning-parametres on the rules. As sensors might have different tuning-parametres, we cannot add the tuning directly to the rules. We rather add the tuning to its own module:

Both filters, and the suppresses is connected to a Rule and a Sensor. This way we let the user spesify filters for the individual sensors if that is desired. If a sensor does not have a filter applied, but any parent of the sensor have one, the closest parent's filter will be applied.

Update

When rules are inserted to the database, we would like to have some traceability to be able to determine from where a rule have entered our system. To be able to do this, we have the update-module:

The structure of the database to update would let us store the update-sources, and add a update-object for each update ran from a given source. These update objects can in turn have a relation to rules, rulesets and rulerevisions, so that we can track what changes for each update.

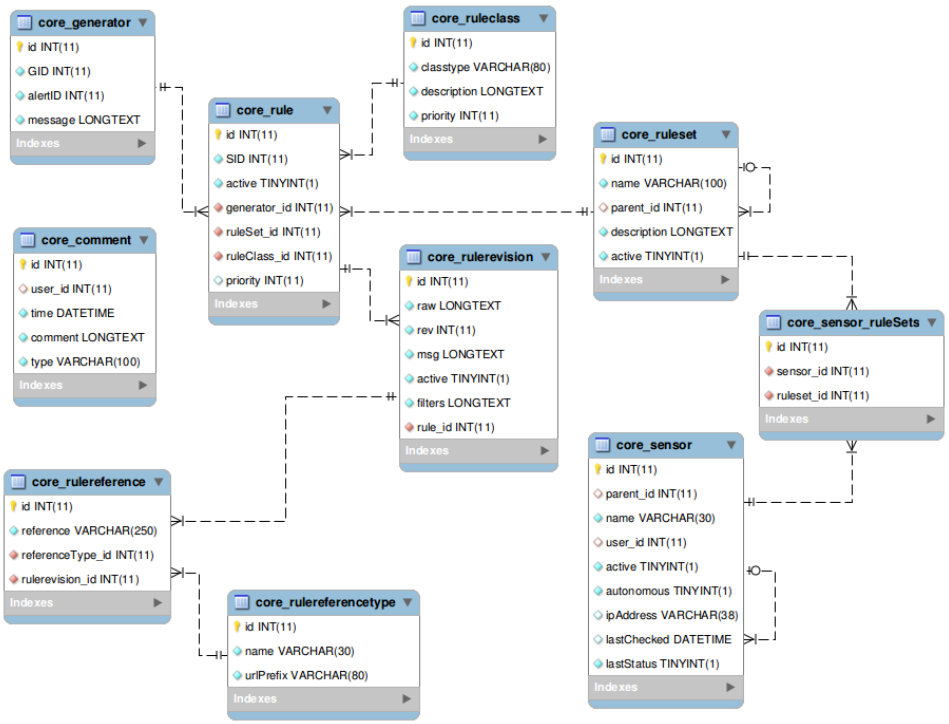


Figure 6: Core server-database

4.2 Client

As the client is just going to take care of getting rules for a single sensor from the central server, and should just apply these to one sensor, the architecture of the client can be much simpler than the one of the server.

4.2.1 Architecture

The MVC schema for the client would look like this:

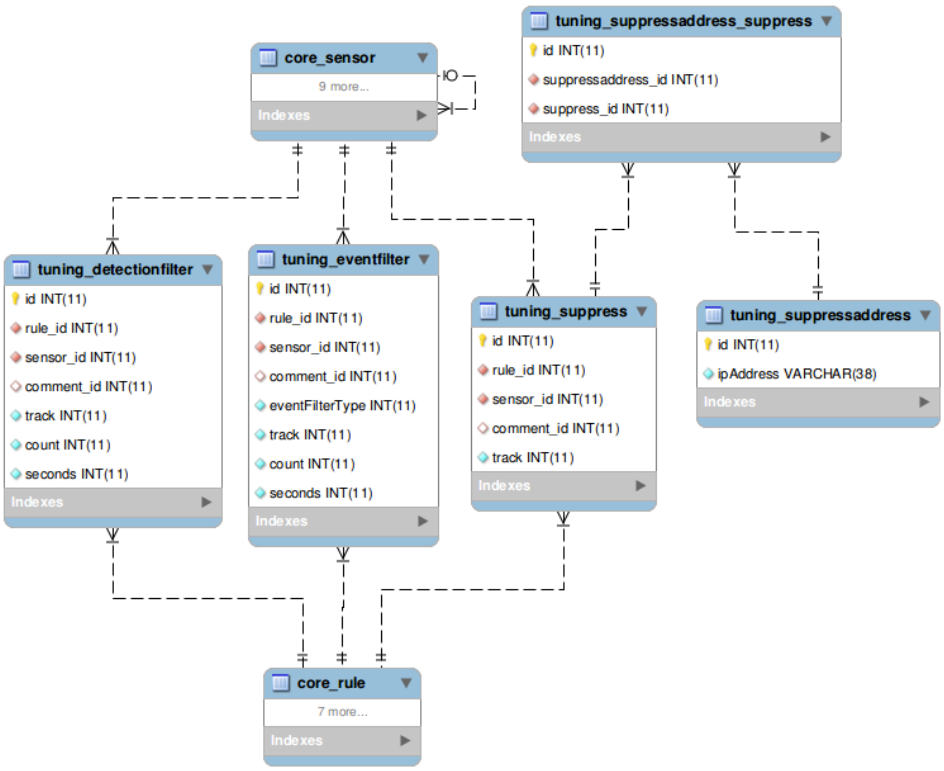


Figure 7: Tuning server-database

./img/Client-MVC.png

Figure 9: Client Architecture

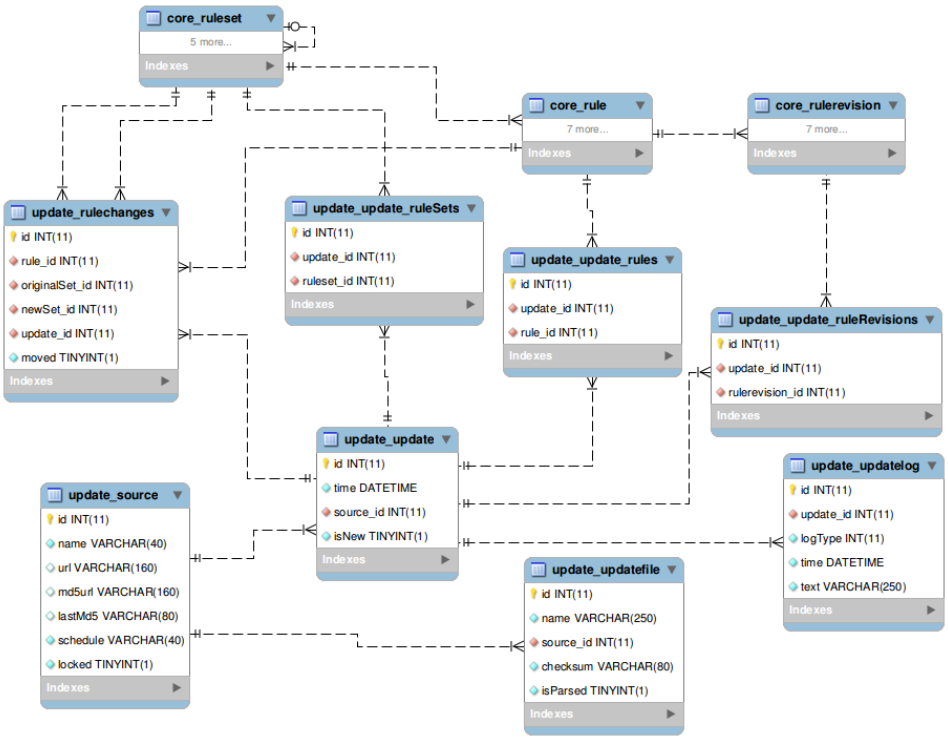


Figure 8: Update server-database

The update-handler of the client have both a XML-RPC Client and a XML-RPC Server. The client is used to connect to the central server, to do a rule-synchronization. The rpc-server is here so that the central server can connect to it and check that the client is alive, or to order an update of the rule-sets.

As the client should be as light-weight as possible, and the only thing we would like to have a framework for is the database-connection, we have decided to not pull inn Django for the client. Instead we use the database ORM for python called "SQL-Alchemy".

4.2.2 Database

The database on the client would be significantly simpler than the one on the server, as we only care about the rules which should be present on a given sensor.

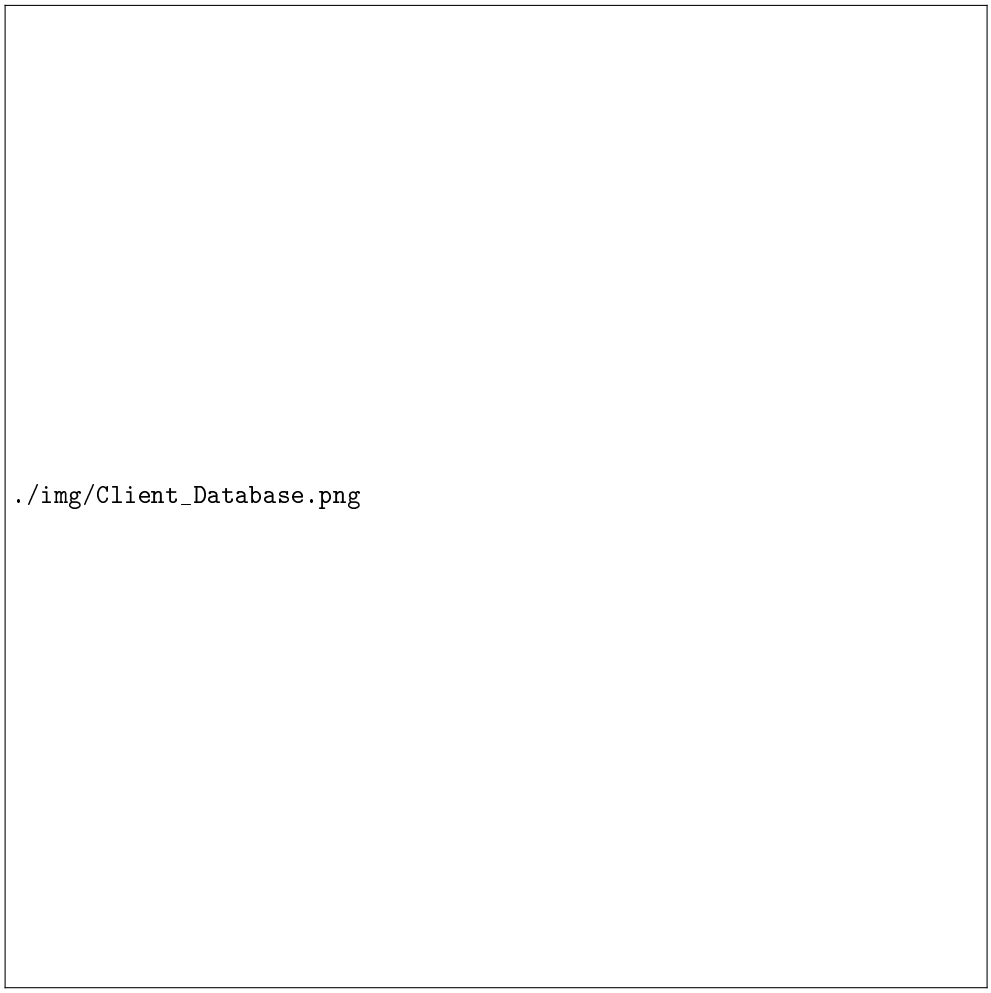


Figure 10: Client Database

The central part of the datastructure is the rule table. This table contains all the parametres directly related to the rule. In addition we have seperate tables to all the parametres modifying the rule (like filters), or that is common between several rules (like rulesets).

5 IMPLEMENTATION

This chapter will focus on the technical implementation of Snowman. The first sections cover programming languages, development tools and frameworks used in this project. To give a better understanding of the implementation, a short introduction to key concepts of Snort has also been included.

5.1 Programming languages

Short description of programming laguages used.

5.1.1 Standards and guidelines

Coding styles, use of exceptions etc.

5.2 Development environments

Short description of development environments and tools.

5.3 Frameworks

To facilitate database transactions and the creation of the Web interface, this project uses the Django[1] framework. Django is a Web framework which takes care of database modelling and transactions, and user authentication.

5.4 Concepts of Snort

This section will give a brief introduction to the basic concepts of Snort.

5.4.1 Rules

A rule is basically a set of options that define ways of detecting, classifying and reporting a potential attack or threat on the network[2]. There are several types of options, but only the general rule options are relevant for this project. The general rule options provide information about the rule [5] (i.e. revision number, alert message). Other types of options, such as payload options which are actively used in detection, are not relevant for the management of rules and are thus not considered in this project.

A rule can either be enabled or disabled in the detection system, meaning that the rule will only have an effect on detection when it is enabled. Since this project is dealing with multiple sensors connected to one central database, this functionality has been expanded so that users can keep a rule enabled on one sensor and disabled on another. In addition, the user can also set an enabled/disabled flag on *rule sets*.

5.4.2 Rule sets

The term rule set, sometimes ruleset, is not formally defined in the Snort Users Manual, however it is used a few times to refer to the complete collection of official Snort rules[3] and to the complete set of rules applied on a Snort system. In practice, rules within a

[1] http://www.djangoproject.com/
[2] A rule actually consists of a header and options [5]
[3] Also referred to as the Sourcefire VRT Certified Rules, see http://www.snort.org/snort-rules/

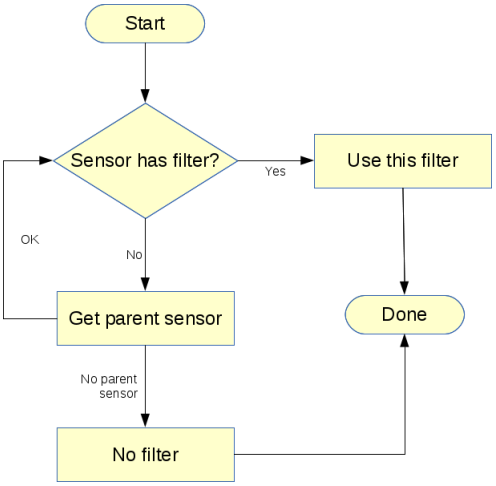


Figure 11: Flow

rule set are categorized by intrusion type (e.g. malware, trojan and worm). These groups of rules are often named by their intrusion type, and are also referred to as rule sets. Snowman and this report use only the latter definition of rule sets.

5.4.3 Generators

Generator is a term used for any subsystem of Snortwhich generates alerts from rules [5]. Each generator is identified by a unique generator ID. The rules subsystem, which has generator id 1, is the only relevant generator for this project.

5.4.4 References

Snort contains a plugin which allows rules to contain references to external attack identification systems which provide additional information about the alerts [5]. A reference is basically a URL string pointing to a webpage containing this information.

5.4.5 Event and detection filters

Filtering can be applied to rules in order to control the amount and frequency of alerts. Two types of filters can be applied to a rule: event filters and detection filters. A detection filter can only apear inline, whereas an event filter can only appear as a standalone option[4].

All filters operate with a *count* and *seconds* variable, representing the number of events in a time interval and the duration of the time interval, respectively. Event filters are further divided into three types (table 10).

5.5 Core module

This section will focus on the objects in the core module. This module contains the most vital objects in Snowman, including rules, rule revisions and sensors. Simple objects such as *generators*, *rule references* and *classtypes*, which are connected to rule objects, are not described in this section. For an overview of all objects and functions in this module, refer to class diagram (ref) or appendix (ref).

[4]An event filter can currently be written inline (called a rule threshold), but according to the Snort Users Manual this feature is deprecated and will not be supported in future releases.

5.5.1 Rules and rule revisions

Rules in Snowman are split into two parts; the rule and the rule revision. As threats change, or errors in the rules are discovered, a rule may become obsolete and a new revision of the rule is released. The rulestring, message and detection filter might change from one revision to the other, and are stored in the rule revision object along with the revision number. A revision is always connected to a rule object which contains data that is constant for all its revisions, such as signature ID and rule set. The reason behind keeping rules and revisions in two separate objects is to avoid writing the same constant rule data for all revisions. Users of Snowman have the possibility to control how many revisions to store per rule, and whether incoming revisions should be active or inactive by default. This feature allows users to control automatically downloaded rules before they are put into the detection system. Users can also manually activate or deactivate rule revisions, and Snowman will distribute only the newest active revision of a rule.

5.5.2 Rule sets

As previously stated, the categorization of rules in rule sets is a de facto way of organizing rules. Rules in the community rule set[5] have a "ruleset" option within the "metadata" option in the rulestring which specifies the rule set. In other releases, rules are organized into files where the file name denote their rule set. These two ways of specifying the rule set are both supported by Snowman.

5.5.3 Sensors

5.5.4 Comments

The core module contains a comment object which can store comments from users who wish to document their operations, e.g. the user can attach a comment when adding a filter to a rule to describe why the filter was added and which effects it will have on the system. Snowman also generate comments for important events, such as ...

5.6 Update module

The purpose of the Update module is to fetch and read rules and configuration files from external sources, and populate the database with this information. Effectiveness in updates is a key issue, and a number of deliberate optimizations for performing updates, with the goal of saving time by omitting unnecessary operations, are explained in the

[5] "The Community Ruleset is a GPLv2 VRT certified ruleset that is distributed free of charge without any VRT License restrictions" - http://www.snort.org/snort-rules/

Filter type	Alerts
Event filter	
Limit	One for every event when number of events $\leq$ C in interval S
Threshold	One for every C events in interval S
Both	One per interval S when number of events $=$ C
Detection filter	One for every event when *count* $>$ C in interval S

Note: C and S represent the count *and* seconds *values in the filter.*

Table 10: Effects of different filter types.

Data field	Description	Presence
sid	Unique identifier for a rule.	required
rev	Rule revision number.	required
msg	Notification text when rule triggers.	required
classtype	Connects the rule with a specific class of attacks.	required
gid	Generator ID to identify which subsystem generates the event where this rule fires.	optional
ruleset	The ruleset this rule belongs to.	optional
priority	Override default severity set by classtype.	optional
reference	Reference to external resource with information about this attack.	optional

Note: The *presence* column indicates whether a field is required or not by the parser in this system. Information is gathered from the Snort Users Manual [5](ch 3.4).

Table 11: Rule options used in this system.

following sections.

The term Update is used to refer to the actual module, including all its classes and functions. To avoid confusion with the module, the term *update-package* is specifically used to refer to update data coming from external sources.

This section will define an example rule to be used throughout subsequent sections and the remainder of the chapter, to illustrate the lifespan of a rule in the system.

Word: rulestring.

5.6.1 Sources

A *source* is used by Update to determine where the update-package is located. Normally this is on a remote site, and the source contains the URL pointing to the location of the update-package. A source can be configured to update at regular intervals, or the user can run an update manually on a source. To make the update process more effective, Update makes use of an MD5 URL which points to a file containing the MD5 hash of the update-package. This hash is stored for every source (if available) and is compared to any pre-existing hash to verify if the system is up to date or not. Update will only download and process an update-package when it encounters a new hash.

5.6.2 Files

An update-package may contain various files depending on the source it comes from. This section will describe files supported by Snowman.

Rule files

A rule file is any file carrying the *.rules* file extension. These files contain raw rulestrings to be processed by Snort. A rulestring is basically a semicolon-separated list of rule options. Specific rule options used in this system are listed in table 11.

A badly formatted rulestring given to Snort will cause a crash, and Snort must be restarted.

Classification, Generator and Reference Configuration files

The classification files are identified by their name *classification.config* and contain rule classification strings that are used by Snort to categorize event data. A classification file will normally appear once per update-package, and rarely changes between updates[6].

Sid message files

5.6.3 Running an update

As an example scenario, consider a rule (name, figure reference, source, ruleset etc.). An update is triggered on the source at the predefined time

The update is cancelled if the source is already up to date; the MD5-hashes of the latest update in the database, if any, and the latest update on the remote site are compared, and the process is terminated if these hashes are identical. Otherwise, the update is downloaded to a temporary directory and the update process continues. A file can be a tar-archive containing an arbitrary number of rulesets and configuration files, or it can be a plain text file containing Snort configuration (?).

The tar-archive is extracted, and the update module recursively parses each file in each folder. In most cases the update-package will contain only minor changes in relation to its predecessor, e.g. only one file is different in the latest package, and parsing all files that has not changed would therefore be a waste. To address this issue and save processing time, the name, path and MD5-hash of every file is stored in the database per source. Every time the update module parses a file, it first creates a hash of the file and compares this with the respective file's hash in the database. If the file is present in the database and the two hashes coincide, the file is considered *not new* and is skipped. The following sections will elaborate on the parsing of files based on file type.

parse files with regex.

5.6.4 Rule parsing

Some options can be defined both in the rulestring (inline) and in a separate file (standalone).

If any properties classified as *required* for a rule (table 11) are missing...

Non-existing rule sets, rule classes and generators are stored in the database

After the initial parsing of the rulestring, the parser checks if the rule already exists in the database, and if so whether the incoming revision number is higher than the latest revision of the pre-existing rule. The parsing will continue only in the event of a new rule or a newer revision of an existing rule.

rule moved to new ruleset.

Rule sets

A rule always belogs to a ruleset which can be specified in the raw rulestring. If the parser does not find this attribute in the rulestring, the filename is considered to be the ruleset name.

Threshold

A rule threshold defines a limit and/or threshold for the triggering of a specific rule, and can exist as an inline or standalone option. Rule thresholds are deprecated, and support for this functionality will cease in a future version of Snort, where it is replaced by *detection filters* and *event filters* (see below) [5]. To maintain compatibility with current

[6] https://www.snort.org/vrt/snort-conf-configurations/

rules, Snowman supports reading inline and standalone threshold definitions.

Detection filters

Please explain..

Event filters

Please explain..

Suppression

Always standalone.

5.6.5 Classification, Generator and Reference Type parsing

These files contain a sequence of their respective entities (section 5.6.2), and are all parsed in the same manner. If a given entity is not already present in the database, it is created and stored. If the database already contains the entity, the existing entry in the database is overwritten.

Rule Message files

5.6.6 Logging

in Snowman this is translated to one filter of each type per rule per sensor.

If a filter occurs inline, this filter is regarded as a default filter and will be active on every sensor where the rule operates. A user can choose to override the default filter on a sensor by defining a custom filter to be used on the respective sensor.

To stop Snortfrom adding duplicate filters, all inline filters are removed from the rulestrings. Any filters extracted from the rulestring are written as plain text (i.e. as they appear in the original rulestring) in the *filters* field of the rule revision object. This is to ensure that all the original rule options are stored meaning that no data is lost or discarded.

Filters are removed from the rulestring and transformed into event filter and detection filter objects.

5.7 Graphical User Interface

The Graphical User Interface (GUI) is a template- and AJAX-based[7] web application which uses Django's template system to implement the View part of the MVC-principle**??**. Each webpage is a template populated through Python-code before it is delivered to the Web server and ultimately the requesting Web browser.

Additional data that needs to be sent to a Web page, e.g. the user interacts with the GUI, is requested through JavaScript[8] and AJAX-calls and then gets added to the already loaded DOM[9] in the web browser.

5.7.1 Structure

The pages of the GUI is organized in the following structure, which is very similar to the way BHTB organized it's front end.

[7] AJAX is web development technique that allows you to send asynchronous requests to a web server.
[8] A programming language commonly used for client-side scripts.
[9] The webpage content or elements.

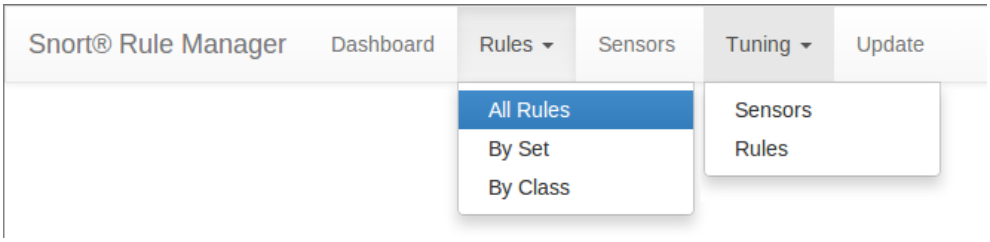


Figure 12: GUI Structure

5.7.2 Design

The GUI is designed with simplicity in mind, which is achieved by using a front-end framework called Bootstrap[10]. This framework provides easy-to-use and ready-made design elements for HTML and CSS[11], which has been crucial for saving time and energy in the project. Consider the example job of creating a <table> element and spending hours setting up and tweaking the CSS for the said table element, we could just add <table class='**table table-bordered**'> and get the table formatted nicely with colors and fonts that are typographically matching.

5.7.3 Overview of central GUI features

Content Prefetching

One of the main features of our GUI design is our usage of prefetching to speed up loading. Instead of loading absolutely everything into the web browsers DOM, like our predecessor BHTB did, we instead initially only load the first page or page elements. This significantly reduces loading times.

We then silently in the background load in any element the user might want to see next, through AJAX calls. When the user then interacts with the GUI, the next elements are shown to the user. The next-next elements are also loaded through AJAX calls so that they are ready.

This ensures that the user always has immidiate response from any interaction done with the GUI, while keeping loading times to a bare minimum.

Rules page

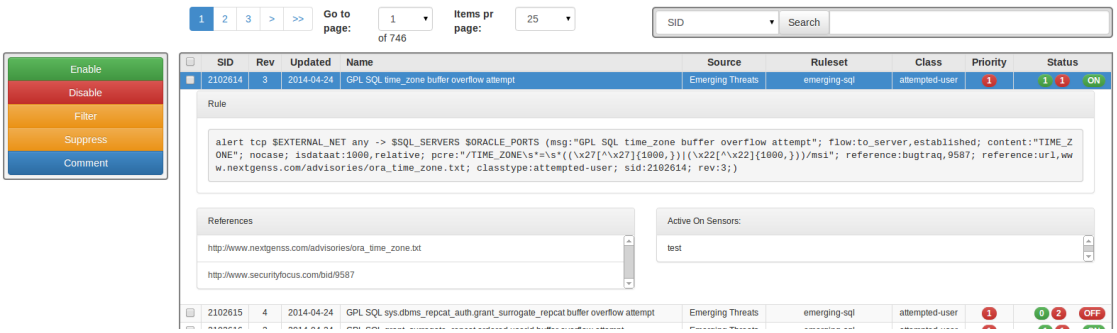


Figure 13: Rules page

[10] http://getbootstrap.com/
[11] A styling language for webpages.

This page serves as the complete list of all rules currently residing in the central database. Because the central database can literally contain hundreds of thousands of rules, this list is paginated. Pages are loaded using AJAX calls to prefetch them as needed.

Each row in the list can be clicked on, which expands an area containing additional information about the rule in question.

We anticipate that the main use of this page is to utilize the search function to find a specific rule or rules and then either retrieve information about them or otherwise manipulate them.

Rules can be tuned by checking the checkbox associated with the rule in question and then pressing the button that represent the desired function in the box to the left.

Rulesets page

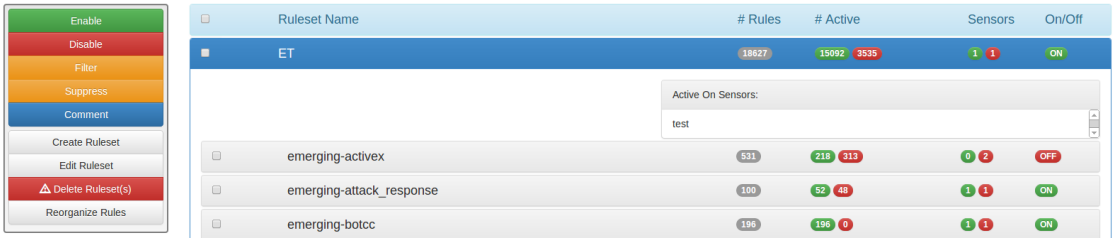


Figure 14: Rulesets page

The rulesets page consists of a list of all the rulesets stored in the central database. Each row contains meta information about the ruleset, such as the number of rules in the ruleset and so on.

A special feature of the ruleset page is that since Snowman allows you to structure rulesets in an hierarcical tree, the ruleset list will initially only show the tree roots. If the ruleset has child rulesets associated with it, those will be listed once you click on the ruleset and open up the additional information area.

The list controls also contain functionality for creating, editing, deleting and reorganizing the rulesets, as well as the ability to turn them on or off either globally or on a per sensor basis.

We also have a very similar page for the sensor list, which also allows you to arrange sensors in a hierarchical tree. Because it is virtually the same, it is not detailed in the report.

Updates and changes

One of the big requests from the client when it came to improvements of features of BHTB was the need to have more feedback over the update process, as BHTB only told you about the status after it was done, leaving the user in an information-limbo.

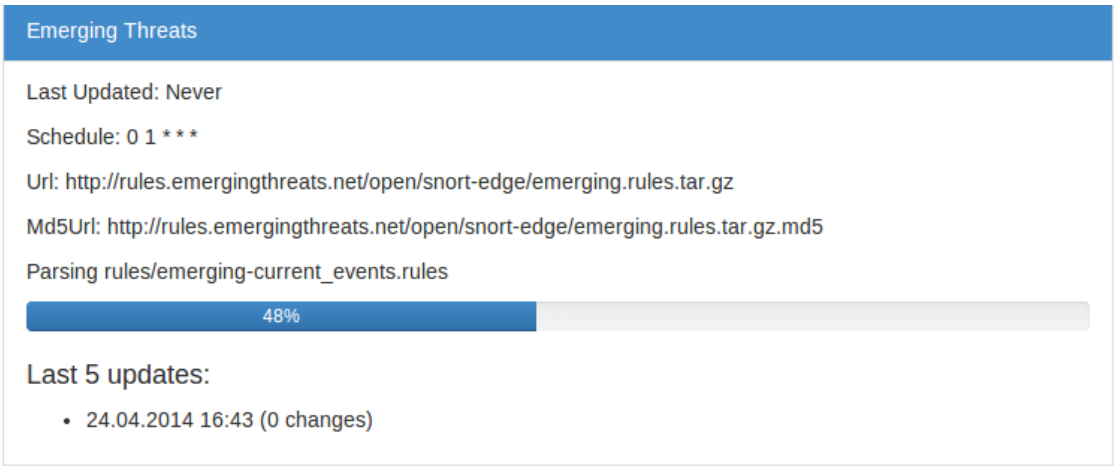


Figure 15: Update progressbar

To solve this, we added a continually updating progressbar which tells the user what the system is currently doing and giving a rough estimate of the percentage of completion.

Another requested feature was the ability to see what was changed during an update. We provide a list for this which contains each update and it's various changes.

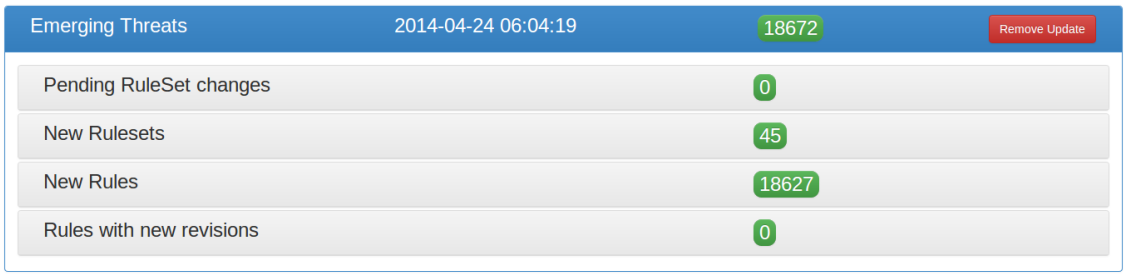


Figure 16: Update changes

The list details any new rulesets, rules or rule revision an update might have brough with it, as well as a list of rules that have been moved from one ruleset to another. Once the user has finished checking out the changes from one update, the user can remove the update from the list. This makes the list a backlog tool, rather than a perpetually long list or some sort of time limited list (say, updates in the last 7 days or something similar). This gives the user a good overview of what has changed since the last time the user checked the list.

Form validation and feedback

One challenge that always face web frontends are forms, as they are a means to alter data in the database and thus become a security challenge. So when you utilize forms, it is important to protect the database against both malicious attackers and user errors.

There are three basic steps to implement protection against these security issues, which is to (1) validate the data provided by the form, (2) make sure the user can't input something it shouldn't and (3) to never let any raw strings or values from the form be directly inserted into a database query.

The third protection is perhaps the most important as it leaves you vulnerable to SQL injection attacks, which are attacks where SQL code is input into the form, letting the attacker carry out his/her own queries, leading to total database compromise. This problem is solved through what is called prepared statements, which makes sure a query doesn't do anything it's not supposed to before the query is actually carried out on the database. This protection is implemented in our system through Django's internal protection mechanisms.

The first and second protection is something we have implemented ourselves at both the frontend and the backend. We make sure that the user can never input invalid values into any form by having Javascript validate the input. So if a user inputs characters where only numbers are allowed, the user will be notified immediately and not allowed to submit the form until it is corrected. And this is done before anything is ever send from the browser to the server.

But if the user has input all valid values, we still validate all the data at the backend

before we store anything in the database. This means checking that any referenced object ID actually exists or making sure the provided IP address is actually valid (within the 0.0.0.0/0 - 255.255.255.255/32 range). If anything is out of place, a message is sent back to the user about it.

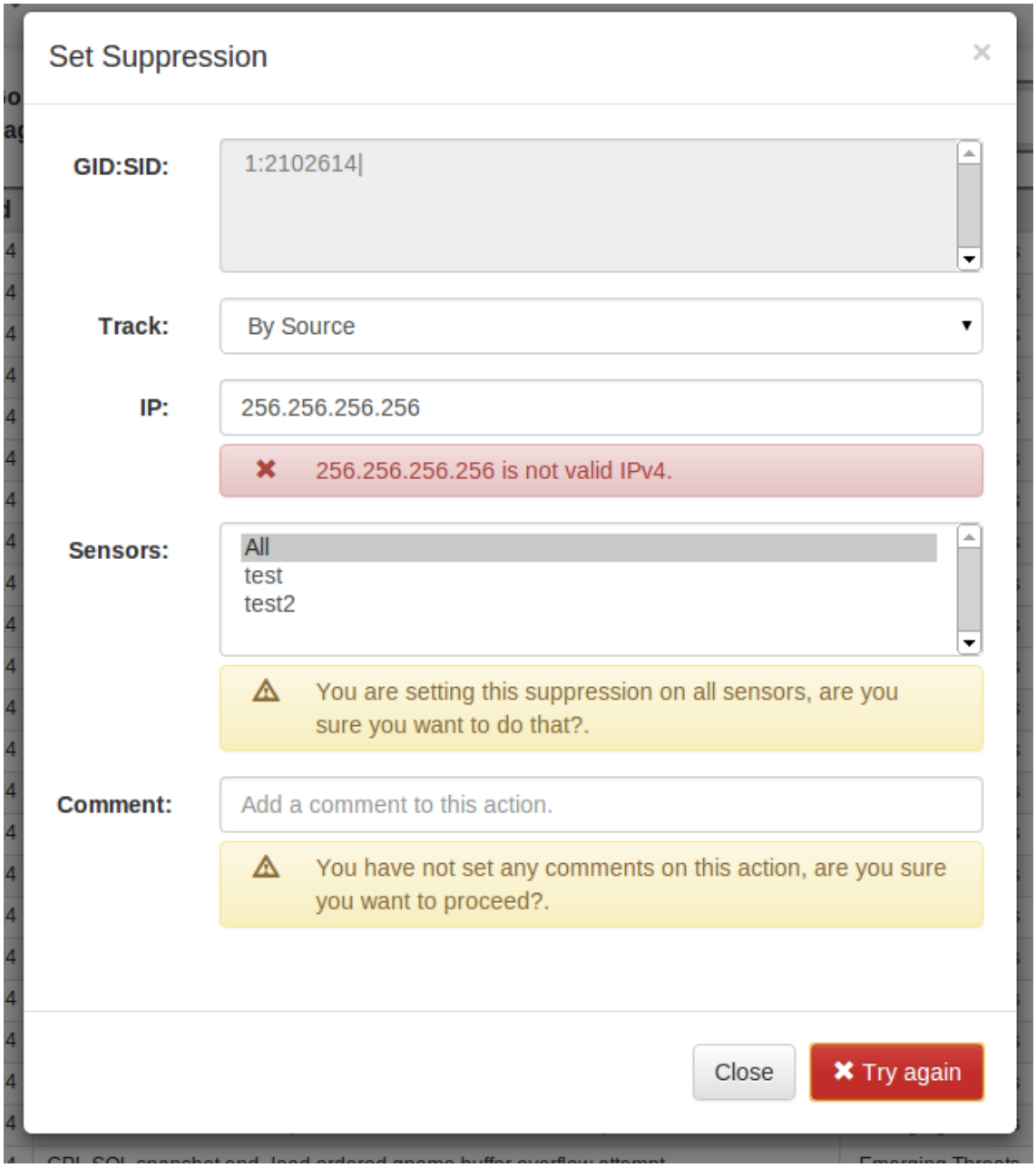


Figure 17: Form validation

5.8 Logging

Describe how logging is implemented in the software.

5.9 Configuration file

Describe the purpose and structure of the config file.

5.10 Documentation

How the software is documented.

6 SOFTWARE TESTING

Her beskrives hvordan opplegget rundt testing av applikasjonen har vært. Dette omfatter planlegging, valg av teststrategier, gjennomføring av tester og beskrivelser av sentrale feil funnet under testing, samt måten feil ble rettet opp på.

6.1 Strategy

The overall strategy of the project when it comes to testing has, first and foremost, been to have regular meetings from the client where we demonstrate live demo's of the software. This has lead us to either confirm things as feature complete or created a list of things to either fix or improve, along the way.

We have also not had any specific regimes when it comes to bugfixing, we have mostly just handled them when they have been discovered them.

You could describe our software testing strategy as ad-hoc based. Our main focus has been on the performance testing.

6.2 Description of problems

put inn noen av problemene vi har hatt her

6.2.1 List pagination

One problem that has plagued the software throughout the development has been the pagination functionality in the GUI.

7 PERFORMANCE TESTING

Since Snowman is similar to BHTB and Snortmanager, it has been important to benchmark their performance in various formal test cases in order to highlight the strength and weaknesses in each software solution. The purpose of the tests is to reach final conclusions on key questions regarding the performance of Snowman, and to point out where the difference between the three software packages lies.

To get a picture of how the software performs on different systems, it was tested on three architectures (table 12). To minimize error, each test was run five times where the mean value of the runs is considered the valid test result. All tests can be reproduced with the test procedure documentation in appendix.

7.1 Definitions

The testing is split up into *test suites*, which contain a *problem* in the form of a question. The problem forms a clear description of what is to be tested, and how results should be evaluated. A test suite also contains one or more *test cases* which exist to address the problem. Test cases represent individual tests, and describe the input data and constraints. Test cases are carried out on the different systems, and results from each test case are reffered to as the *test results*.

7.2 Environment

All tests are carried out on three systems, named *S1*, *S2* and *S3* (table 12). These systems differ in processing power, available memory and storage technology, and the results will indicate what impact these factors have for software performance. The performance testing in this report is also prone to error, as tests might get disturbed by other background processes etc. Testing on three systems gives results with a higher confidence, as results are checked aginst each other for inconsistencies. When an inconsistency in results is found, the test is run again to see if the inconsistency prevails.

System	Processor type	Memory	Storage
S1	Intel® Core™ 2 Solo, 1.4GHz	4GB RAM	5K RPM HDD
S2	Intel® Core™ i7, 6x3.2GHz	12GB RAM	10K RPM HDD
S3	Intel® Core™ i5, 4x3.2GHz	16GB RAM	SSD

Note: See appendix for full details.

Table 12: Test systems.

7.3 Test suites

Even though Snowman, BHTB and Snortmanager share the same goal of being a rule management system, it is not immediately obvious how they can be tested against each other to compare their performance. The first step in the performance testing has been to

identify the areas where the three software packages share, to a certain extent[1], the same functionality. In the test planning, considerations have been made to ensure that the tests hold an acceptable degree of fairness (i.e. tests are designed so that none of the software enjoys any improper advantages or disatvantages), and that the tests are pertinent to the daily operation of the software and the Snowman requirement specification.

Snowman, BHTB and Snortmanager all have the functionality to download and process an update from an external source. The first group of test suites presents different test cases to benchmark how each software performs in update processing.

7.3.1 Update processing

The purpose of the tests in this section is first and foremost to test how quickly the software can process an update, and if the processing time is linear to the update size. It is interesting to test for linearity as this may help point out programming errors. To get a complete picture of the performance, the tests are set up with different constraints such as update size, number of files and number of changes from one update to the next.

Tests numbered 1 through 4 use an artificially generated update package where all rules contain the same data[2].

Test 1

Problem: How fast can the software process an update?

The purpose of this question is to determine the average processing time per rule. Results are ranked from short to long processing time, where shorter is better.

Test cases:

To address this problem, five test cases were introduced. The tests contained an artificially generated update with 1, 10, 100, 1000 and 10000 rules, respectively. Rules were split into files with ten each (table 13). Each test case represents one update, and the names denote the number of rules, files and rules per file in the following structure: *rules-files-rules per file*.

Test case	Rules	Files
1-1-1	1	1
10-1-10	10	1
100-10-10	100	10
1000-100-10	1000	100
10000-1000-10	10000	1000

Table 13: Test cases in Test 1.

[1] Snowman, BHTB and Snortmanager have been built to tackle many of the same challenges, however, the actual implementations may differ severly. For an extended walk-through of differences in implementation, refer to chapter 2, and chapter 9.

[2] Only signature IDs are different. The rules contain a message and a classtype, see appendix for full description.

Test 2

Problem: How strong is the linear correlation between processing time and the size of an update?

This question rises from the requirements specification of Snowman (chapter 3), where it is stated that Snowman shall exhibit a linear increase in processing time in correlation to the number of rules in the update (where the number of rules depicts the update size). Results are ranked from strong correlation to no correlation, where stronger is better.

Test cases:

This test uses the same test cases as Test 1.

Test 3

Problem: Does raising the ratio between files and rules increase processing time?

Updates can come with the rules either gathered in one file, or distributed over several files. This question is asked to examine the difference in processing time for these two scenarios. Results are ranked from no difference to big difference, where less difference is better.

Test cases:

This test compares the *1000-100-10* test case from Test 1 with a test case with the same number of rules in *one* file (1000-1-1).

Test 4

Problem: Is the processing time shorter when the update contains no new rules?

This is a test for update processing when the update is not new (i.e. it has been completely processed by the system before). To minimize system load, it is essential that already processed updates are (in practice, they should be neglected) by the system. Results are ranked from shortest to longest processing time, where shorter is better.

Test cases:

This is a compound test where a test case is run two times in succession. The 1000-100-10 test case from Test 1 is run first to populate the database, before it is run again.

Test 5

Problem: Are the processing times for two updates identical when only one rule has changed between the updates?

This question seeks to determine how the software handles an update when there is only one change from the previous update. This is an important factor to consider, since an update usually contains only a small amount of changes from previous updates. Results are ranked from short processing time for second update to no difference in processing time, where shorter time is better.

Test cases:

This test suite consists of two test cases, one with an artificially generated update and one with an update from a real source. The test cases 1000-100-10 and from Test 1 are used. Each update is run twice; the first run populates the database, and the second run contains one change from the previous run (one of the rules has an updated revision number).

7.3.2 Rule distribution

8 RESULTS

This chapter presents the results from the performance testing.

Mean times calculated from five sequential test results. See appendix for a complete listing of results.

- Test 1 and 2, table 14.
- Test 3, table 15.

Software	Test case	S1	S2	S3
Snowman v0.4	1-1-1	1058.179	151.946	622.640
	10-1-10	2289.518	327.655	1668.142
	100-10-10	17519.385	2356.610	12197.192
	1000-100-10-A	167645.267	22830.358	114617.561
	10000-1000-10	N/A	228038.066	$1.162 \cdot 10^6$
	SF-28064-A	$9.175 \cdot 10^6$	$1.267 \cdot 10^6$	$3.817 \cdot 10^6$
BHTB	1-1-1	125.236	18.175	65.281
	10-1-10	148.323	19.711	72.025
	100-10-10	185.970	25.024	84.913
	1000-100-10-A	568.333	60.313	150.538
	10000-1000-10	3606.043	440.334	910.172
Snortmanager	1-1-1	228.617	36.960	93.403
	10-1-10	265.275	37.569	107.554
	100-10-10	1251.275	159.113	139.005
	1000-100-10-A	9470.460	1360.276	491.512
	10000-1000-10	92428.555	13523.887	5105.158

All times in milliseconds. For complete results, see appendix.

Table 14: Test results for Test 1 and 2.

Software	Test case	S1	S2	S3
Snowman v0.4	1000-1-1000	147170.285	20367.838	113901.478
BHTB	1000-1-1000	445.476	49.908	175.805
Snortmanager	1000-1-1000	1836.396	228.044	509.243

All times in milliseconds. For complete results, see appendix.

Table 15: Test results for Test 3.

Software	Test case	S1	S2	S3
Snowman v0.4	1000-100-10-B	296.272	56.152	N/A
BHTB	1000-100-10-B	7.816	1.310	N/A
Snortmanager	1000-100-10-B	1763.480	205.956	N/A

Note: Showing results for the second run of the test case.

Table 16: Test results for Test 4.

Software	Test case	S1	S2	S3
Snowman v0.4	1000-100-10-C	296.272	56.152	N/A
BHTB	1000-100-10-C	7.816	1.310	N/A
Snortmanager	1000-100-10-C	1763.480	205.956	N/A

Note: Showing results for the second run of the test case.

Table 17: Test results for Test 5.

9 ANALYSIS OF RESULTS

Diskusjon rund resultatene. Det skal gjøres en evaluering av hvorvidtvalgt løsning. Avvik drøftes. det er samsvar mellom kravspesifikasjonen satt opp og
An update package usually contains only a small number of changes from the previous version[1].

[1]Ten updates from 20.03.2014 to 17.04.2014 contained on average 36.8 new and 53.9 updated rules. *Source: http://www.snort.org/vrt/advisories*

10 CONCLUSIONS

De slutninger som kan trekkes fra resultatene og deres diskusjon skal være logisk og nøkternt oppbygd. Etterrettelighet er vesentlig. Konklusjoner skal fremkomme klart og uten tvetydigheter. Eventuell usikkerhet påpekes. Grunnlaget for slik tvil skal redegjøres under kapittel 6a/6b. De viktigste konklusjoner blir presentert i kortfattet versjon under ”Sammendrag”. Prosjektgruppen bør ta stilling til hva de faglige resultatene kan brukes til, hva gruppen faglig har lært av dette arbeidet, hva som evt. kunne vært gjort annerledes ved en gjentagelse av utviklingsarbeidet, og hvordan eventuelt videre arbeid rundt samme eller beslektet tema bør gjøres. Gruppen bør også kommentere den subjektive opplevelse av hovedprosjektet, og hvordan gruppeprosessen har vært. Kort sagt: Egenevaluering av gruppearbeidet.

Bibliography

[1] St.Laurent, A. M. 2008. *Understanding Open Source and Free Software Licensing*. O'Reilly Media.

[2] Roesch, M. 1999. Snort - lightweight intrusion detection for networks. *Proceedings of LISA '99: 13th Systems Administration Conference*.

[3] Henriksen, D. O. Managing signatures for ids in a distributed environment - a study of a signature management system. Master's thesis, Gjøvik University College, 2012.

[4] Vaskinn, C. & Wikestad, K. Snortmanager. Bachelor's thesis, Gjøvik University College, 2012.

[5] Roesch, M. et al. 2012. *Snort Users Manual 2.9.3*. The Snort Project.